



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

КУРСОВА РАБОТА

дисциплина: „Програмиране за мобилни устройства”

**тема: „The 100 National Tourist Sites – мобилно приложение за 100-те
национални туристически обекта”**

Изготвили (III курс, специалност КСИ):

Петър Жан Иванов

Фак. №: 121223054

42Б група

E-mail: petarzivanov@tu-sofia.bg

София, 2026

Съдържание

1. Въведение

2. Използвани технологии

2.1 .NET MAUI

2.2 C#

2.3 Android и Android SDK

2.4 MAUI Geolocation API

2.5 MAUI Preferences

2.6 Google Maps връзки

3. Какво беше необходимо, за да работи приложението

3.1 Конфигурация на .NET MAUI проекта

3.2 Android permissions

3.3 Локални данни

3.4 Запазване на потребителски избори

4. Архитектура

4.1 Общ архитектурен модел

4.2 Model слой

4.3 Data слой

4.4 Service слой

4.5 View слой

5. Модел на данните

6. Основни функционалности

6.1 Начален списък с туристически обекти

6.2 Детайлен екран

6.3 Любими обекти

6.4 Посетени обекти и прогрес

6.5 Моята локация

6.6 Най-близки обекти

6.7 Случаен непосетен обект

7. Работни процеси

7.1 Разглеждане на обект

7.2 Добавяне в любими

7.3 Маркиране като посетен

7.4 Намиране на най-близки обекти

8. Описание на реализацията

8.1 Place.cs

8.2 FavoritesService и VisitedService

8.3 LocationPage

8.4 NearestPlacesPage

8.5 Случаен непосетен обект

9. Потребителско ръководство и тестване

9.1 Основни действия

9.2 Тестови сценарии

10. Екрани за демонстрация

11. Трудности и преодоленни проблеми

12. Заключение

13. Литература

1. Въведение

The 100 National Tourist Sites представлява мобилно приложение за разглеждане, запазване и проследяване на туристически обекти от инициативата „100 национални туристически обекта“. Проектът е реализиран с .NET MAUI и е насочен към Android устройства.

Основната идея е потребителят да има удобен туристически гид в телефона си. Приложението показва списък с обекти, снимки, кратко и подробно описание, интересен факт, координати и връзка към Google Maps. Допълнително се поддържат любими и посетени обекти, прогрес и избор на случаен непосетен обект.

Проектът е подходящ за дисциплината „Програмиране за мобилни устройства“, защото използва реални мобилни възможности: GPS локация, Android permissions, локално съхранение на потребителски избори и работа върху реално Android устройство.

2. Използвани технологии

2.1 .NET MAUI

.NET MAUI е основната технология за разработка на приложението. Тя позволява създаване на мобилни и desktop приложения с C# и общ код. В проекта MAUI се използва за изграждане на екраните, навигацията, бутоните, списъците и работата с ресурси като икона и splash screen.

2.2 C#

C# е използван за цялата логика на приложението. Чрез него са дефинирани моделите, страниците, service класовете, филтрирането, изчисляването на разстояния и записването на потребителските избори.

2.3 Android и Android SDK

Приложението е тествано върху Android устройство. За да работи коректно, проектът използва Android SDK, AndroidManifest.xml и нужните permissions за интернет и локация.

2.4 MAUI Geolocation API

Geolocation API се използва за взимане на текущата локация на потребителя. Получените latitude и longitude стойности служат за показване на координати и за изчисляване на най-близките туристически обекти.

2.5 MAUI Preferences

Preferences се използва като лек механизъм за локално съхранение. В проекта чрез Preferences се пазят ID-тата на любимите и посетените обекти. Така данните остават налични след затваряне и повторно стартиране на приложението.

2.6 Google Maps връзки

Всеки туристически обект има Google Maps URL. При натискане на бутон приложението отваря външна карта и позволява навигация към избраната локация.

3. Какво беше необходимо, за да работи приложението

3.1 Конфигурация на .NET MAUI проекта

За реализацията беше необходимо да бъде инсталиран .NET MAUI workload във Visual Studio, заедно с Android SDK. Проектът е създаден като .NET MAUI App и е стартиран върху реален Android телефон.

3.2 Android permissions

За GPS функционалността и зареждането на изображения от интернет в AndroidManifest.xml са добавени нужните permissions.

Код 3.1: Android permissions

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.INTERNET" />
```

3.3 Локални данни

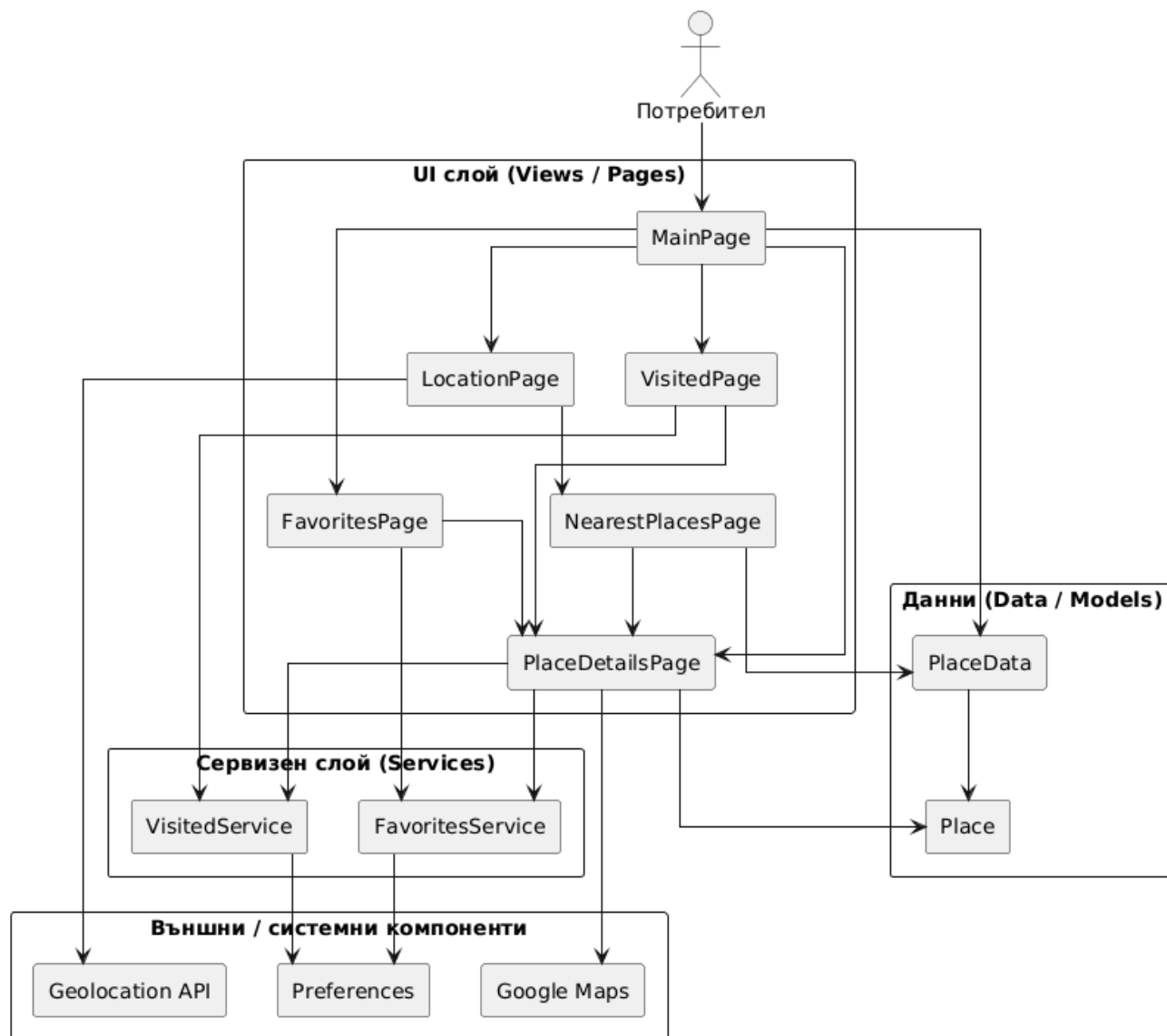
Проектът не използва външна база данни. Туристическите обекти са дефинирани локално в PlaceData.cs. Това решение е достатъчно за курсова работа и позволява бърза демонстрация без нужда от сървър.

3.4 Запазване на потребителски избори

Любимите и посетените обекти се пазят чрез Preferences. Записват се само ID-та, а при зареждане на списъците пълната информация се взима от PlaceData.

4. Архитектура

Фигура 4.1: Обща архитектура на приложението



4.1 Общ архитектурен модел

Приложението използва опростена структура Model + Data + Services + Views. Моделът описва туристически обект, Data слойът предоставя началните данни, Services съхраняват състоянието за любими и посетени, а Views реализират потребителските екрани.

4.2 Model слой

Model слойът съдържа класа Place, който описва всички данни за един туристически обект: име, категория, регион, описание, снимка, координати, Google Maps линк и статуси.

4.3 Data слой

Data слойт съдържа PlaceData.cs. В него е дефиниран списъкът с туристически обекти. Този файл изпълнява ролята на локален каталог.

4.4 Service слой

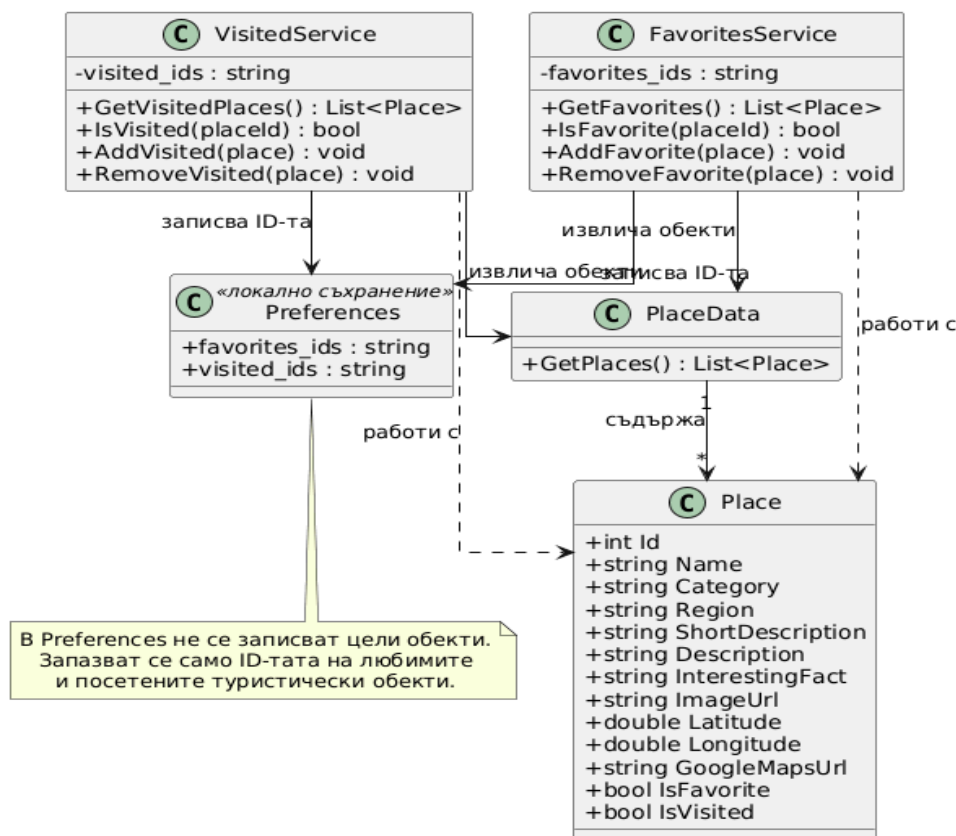
Service слойт включва FavoritesService и VisitedService. Те отговарят за добавяне, премахване, проверка и зареждане на любимите и посетените обекти.

4.5 View слой

View слойт съдържа страниците MainPage, PlaceDetailsPage, FavoritesPage, VisitedPage, LocationPage и NearestPlacesPage. В тях е реализиран интерфейсът, навигацията и обработката на потребителските действия.

5. Модел на данните

Фигура 5.1: Логически модел на данните



Основният модел в приложението е Place. Той съдържа всички данни, нужни за визуализация, навигация и съхранение на потребителски статус.

Поле	Описание
Id	Уникален идентификатор на обекта
Name	Име на туристическия обект
Category	Категория – манастир, крепост, природа и др.
Region	Регион или област
ShortDescription	Кратко описание за началния списък
Description	Подробно описание за детайлната страница
InterestingFact	Кратък интересен факт
ImageUrl	Връзка към снимка
Latitude / Longitude	Географски координати
GoogleMapsUrl	Връзка към Google Maps
IsFavorite / IsVisited	Статуси за любими и посетени

Проектът не използва релационна база данни, защото данните са ограничени и са подходящи за локален каталог. За потребителските действия се използва Preferences, което е по-леко и по-безопасно решение за този тип приложение.

6. Основни функционалности

6.1 Начален списък с туристически обекти

Началният екран показва списък с обекти. Всеки елемент съдържа снимка, име, категория, регион, кратко описание и бутон за детайли. Ако обектът е любим или посетен, това се показва визуално със статус.

6.2 Детайлен екран

Детайлната страница представя пълната информация за избрания обект. Потребителят може да го добави в любими, да го маркира като посетен или да отвори локацията в Google Maps.

6.3 Любими обекти

Потребителят може да добавя и премахва любими обекти. Всички любими се показват в отделна страница. При празен списък се показва съобщение.

6.4 Посетени обекти и прогрес

Обектите могат да бъдат маркирани като посетени. На началния екран се показва броят на посетените спрямо всички налични обекти и процентен прогрес.

6.5 Моята локация

Страницата за локация взема текущите координати на устройството и ги показва на потребителя. Тази функционалност доказва реална работа с мобилен хардуер.

6.6 Най-близки обекти

След взимане на GPS локация приложението изчислява разстоянието до всички обекти, сортира ги и показва най-близките 5.

6.7 Случаен непосетен обект

Потребителят може да поиска случаен обект за посещение. Изборът се прави само от обекти, които още не са маркирани като посетени.

7. Работни процеси

7.1 Разглеждане на обект

1. Потребителят отваря началния екран.
2. Приложението зарежда обектите от PlaceData.
3. Потребителят избира бутон „Детайли“.
4. Отваря се PlaceDetailsPage за избрания обект.

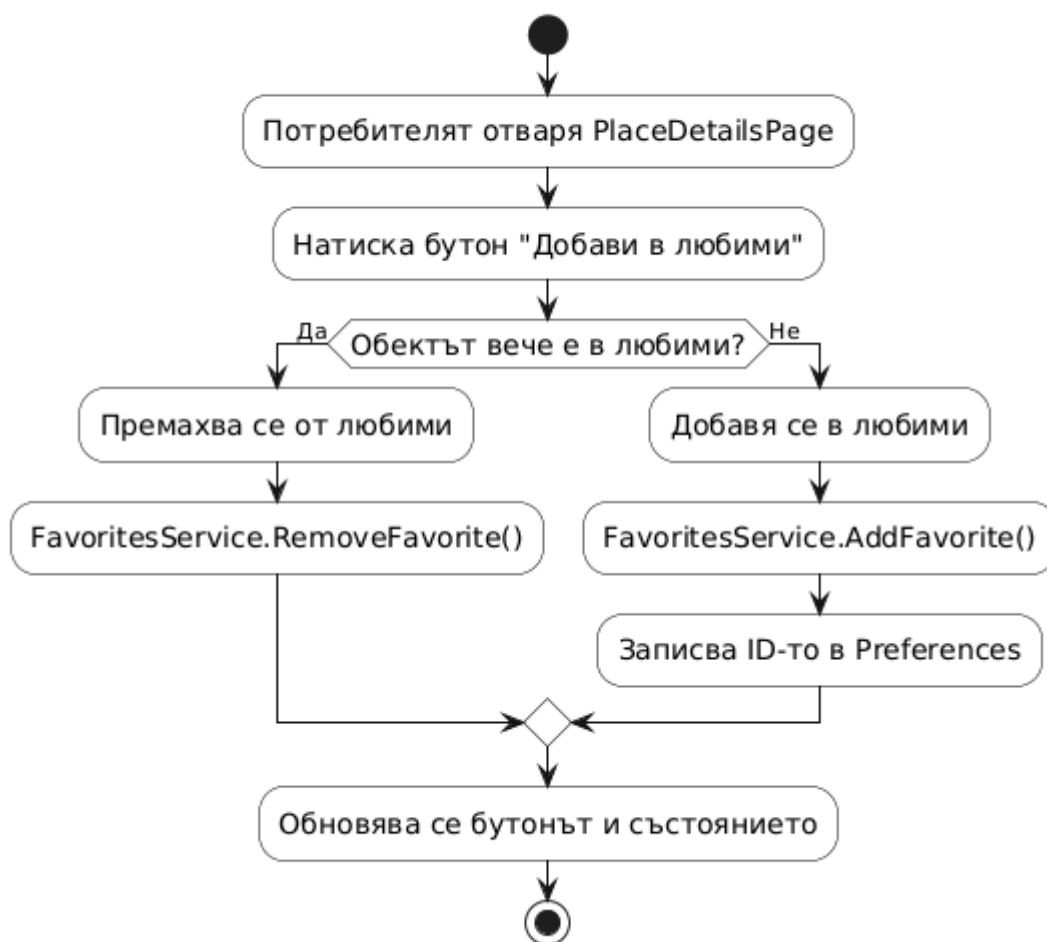
Фигура 7.1: Работен процес при разглеждане на обект



7.2 Добавяне в любими

- Потребителят отваря детайлите на обект.
- Натиска „Добави в любими“.
- FavoritesService добавя ID-то в списъка.
- Данните се записват в Preferences.
- При следващо стартиране изборът се възстановява.

Фигура 7.2: Работен процес при добавяне в любими



7.3 Маркиране като посетен

10. Потребителят отваря детайлите на обект.
11. Натиска „Маркирай като посетено“.
12. VisitedService записва ID-то в Preferences.
13. Началната статистика и прогресът се обновяват.

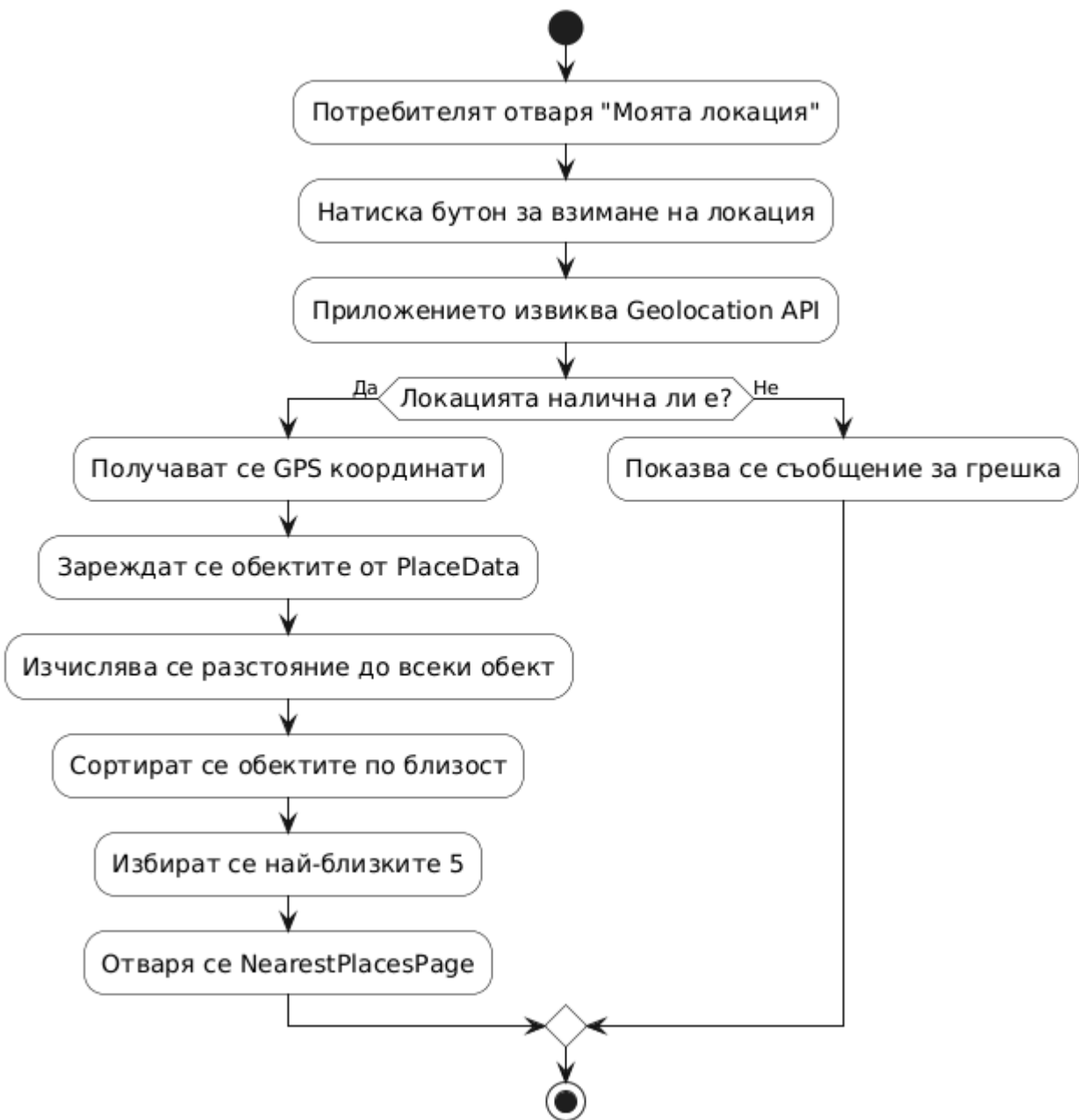
Фигура 7.3: Работен процес при маркиране като посетен



7.4 Намиране на най-близки обекти

14. Потребителят отваря „Моята локация“.
15. Приложението взима GPS координати.
16. Изчислява разстояние до всички обекти.
17. Сортира обектите по близост.
18. Показва най-близките 5.

Фигура 7.4: Работен процес при намиране на най-близки обекти



8. Описание на реализацията

8.1 Place.cs

Place.cs описва структурата на туристическия обект. Класът съдържа както текстова информация, така и координати, снимка и линк към Google Maps.

Код 8.1: Основен модел Place

```
public class Place
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Category { get; set; }
    public string Region { get; set; }
    public string ImageUrl { get; set; }
    public double Latitude { get; set; }
    public double Longitude { get; set; }
    public bool IsFavorite { get; set; }
    public bool IsVisited { get; set; }
}
```

8.2 FavoritesService и VisitedService

Двата service класа използват сходна логика. Вместо да записват цели обекти, те пазят само ID-та. Това намалява сложността и прави съхранението стабилно.

Код 8.2: Записване чрез Preferences

```
var data = string.Join(",", _favoriteIds);
Preferences.Set("favorites_ids", data);
```

8.3 LocationPage

LocationPage използва Geolocation API, за да вземе текущата локация. При липса на последна известна локация се прави нова заявка към устройството.

Код 8.3: Взимане на GPS локация

```
var location = await Geolocation.GetLastKnownLocationAsync();
if (location == null)
{
    location = await Geolocation.GetLocationAsync(
        new GeolocationRequest(GeolocationAccuracy.Medium));
}
```

8.4 NearestPlacesPage

NearestPlacesPage приема текущата локация, изчислява разстоянието до всеки обект и показва най-близките. Изчислението използва радиуса на Земята и координатите на двете точки.

Код 8.4: Избор на най-близки обекти

```
var nearestPlaces = places
    .Select(p => new { Place = p, Distance = CalculateDistance(...) })
    .OrderBy(p => p.Distance)
    .Take(5)
    .ToList();
```

8.5 Случаен непосетен обект

Random функционалността първо филтрира посетените обекти и избира само от останалите. Ако всички са посетени, приложението показва съобщение.

Код 8.5: Избор на случаен непосетен обект

```
var unvisitedPlaces = allPlaces
    .Where(p => !VisitedService.IsVisited(p.Id))
    .ToList();
var selectedPlace = unvisitedPlaces[random.Next(unvisitedPlaces.Count)];
```

9. Потребителско ръководство и тестване

9.1 Основни действия

- Стартиране на приложението от Android устройство.
- Разглеждане на списъка с туристически обекти.
- Отваряне на детайлен екран.
- Добавяне в любими и маркиране като посетен.
- Отваряне на страници „Любими“ и „Посетени“.
- Вземане на текуща GPS локация.
- Показване на най-близките обекти.
- Избор на случаен непосетен обект.
- Отваряне на локация в Google Maps.

9.2 Тестови сценарии

Действие	Очакван резултат	Реален резултат
Отваряне на началния екран	Показва се списък с обекти	Успешно
Добавяне в любими	Обектът се появява в Любими	Успешно
Рестарт на приложението	Любимите и посетените остават запазени	Успешно
Маркиране като посетен	Прогресът се увеличава	Успешно
Вземане на GPS локация	Показват се координати	Успешно
Най-близки обекти	Показват се 5 най-близки обекта	Успешно
Google Maps бутон	Отваря се карта с локацията	Успешно
Случаен непосетен обект	Отваря се непосетен обект	Успешно

10. Екрани за демонстрация

Фигура 10.1: Начален екран



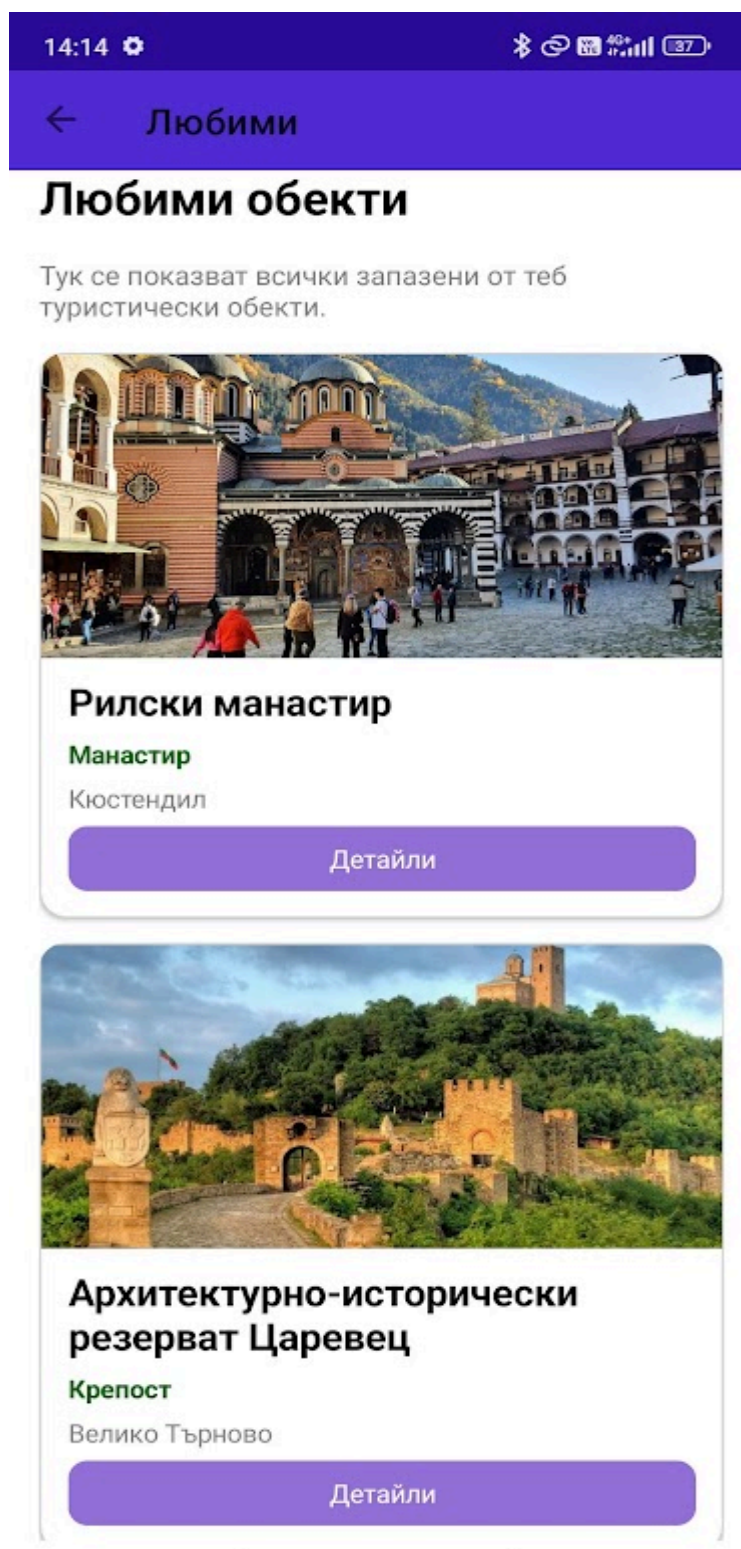
Екранът демонстрира основния списък с обекти, статистика за любими и посетени и бутоните за навигация.

Фигура 10.2: Детайли за туристически обект



Тук се виждат снимка, описание, интересен факт, координати и действията за любим/посетен.

Фигура 10.3: Любими обекти



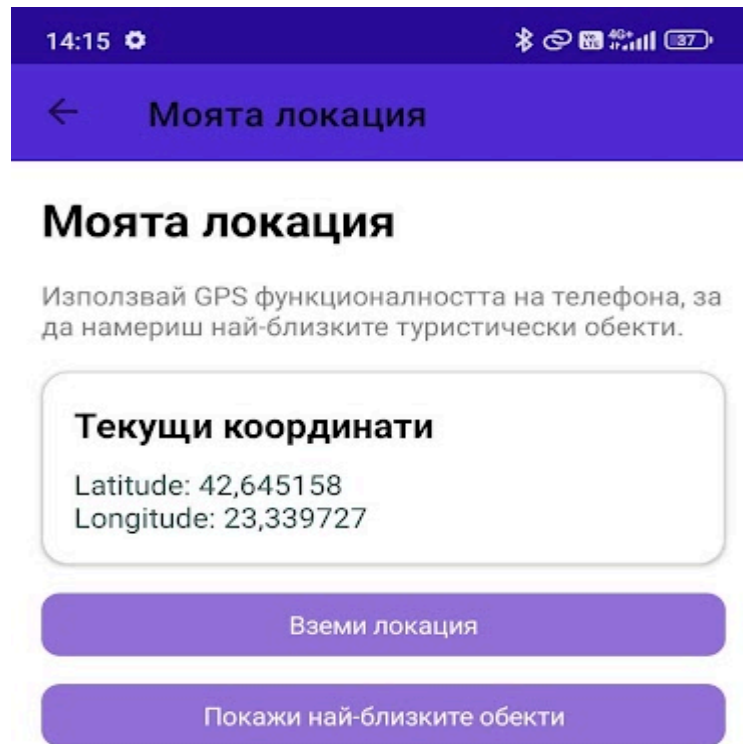
Екранът показва всички запазени от потребителя любими обекти.

Фигура 10.4: Посетени обекти



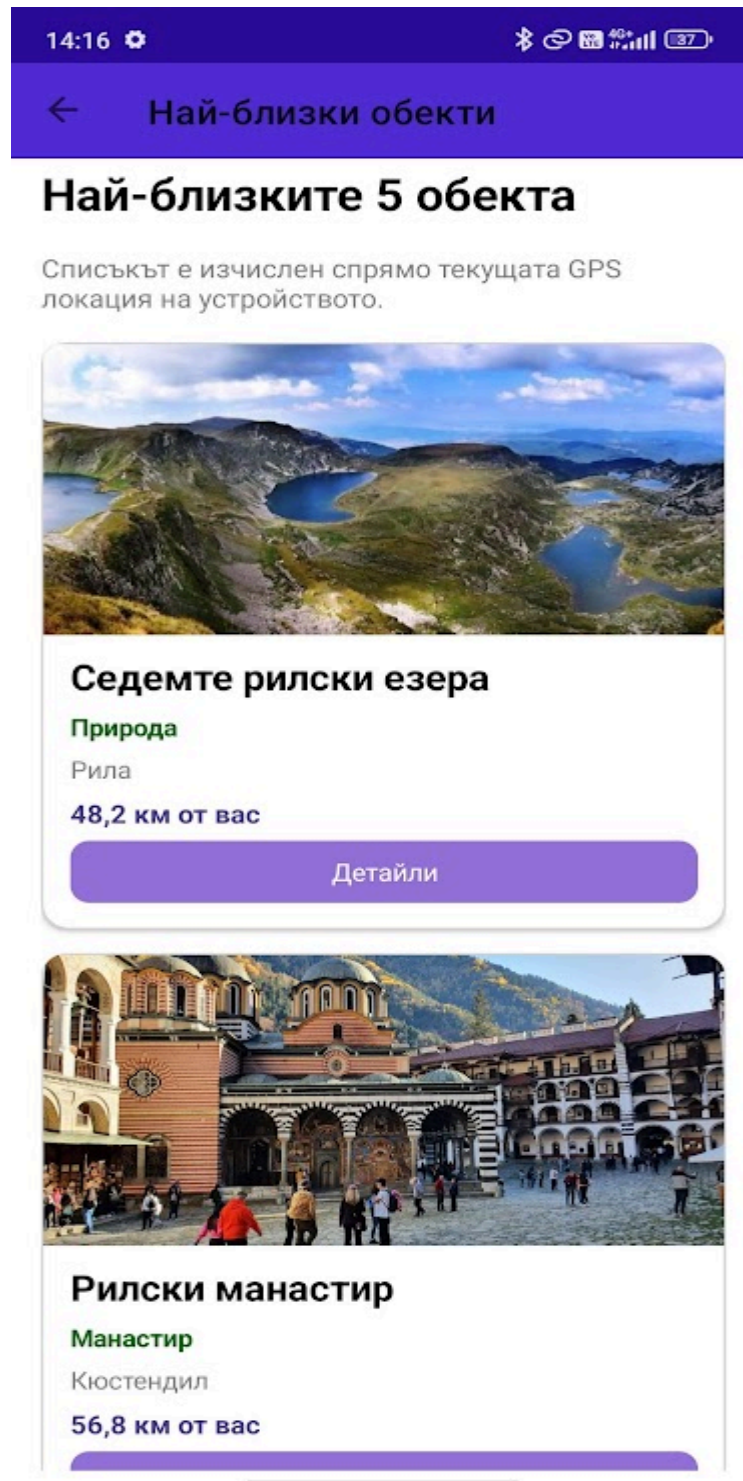
Екранът показва всички маркирани като посетени обекти.

Фигура 10.5: Моята локация



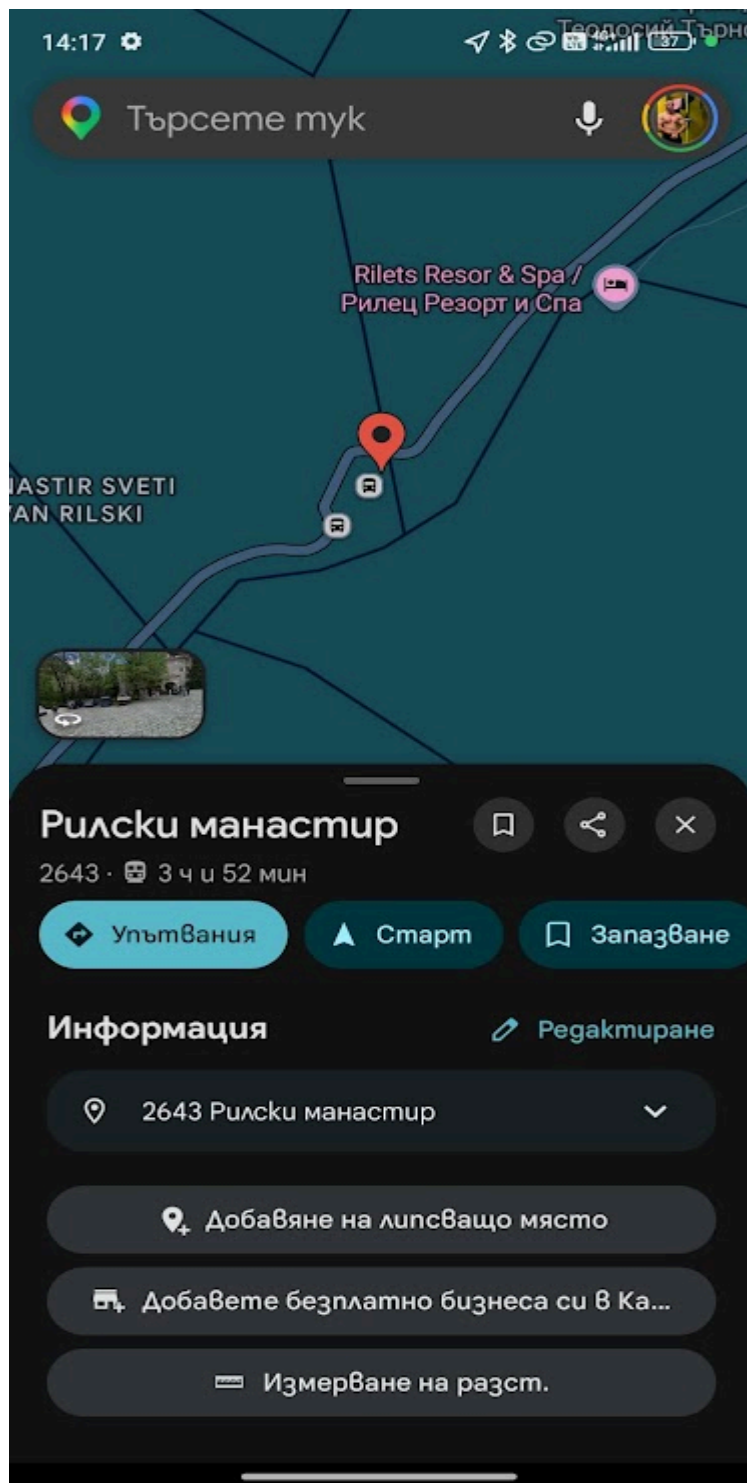
Екранът демонстрира взимането на GPS координати от устройството.

Фигура 10.6: Най-близки обекти



Тук се показват най-близките обекти, подредени по разстояние.

Фигура 10.7: Google Maps навигация



Демонстрира отваряне на избран обект в Google Maps.

11. Трудности и преодоленни проблеми

Първото предизвикателство беше настройването на .NET MAUI средата и Android устройството. За успешна работа беше необходимо инсталиране на правилните workload-и, Android SDK и активиране на USB debugging.

Вторият проблем беше свързан с Android permissions. GPS функционалността не работи без ACCESS_COARSE_LOCATION и ACCESS_FINE_LOCATION, а изображенията и външните ресурси изискват INTERNET permission.

Друго важно решение беше изборът между SQLite и Preferences. За текущата задача Preferences е по-подходящ вариант, защото трябва да се пазят само малки списъци от ID-та. Това намали риска и осигури стабилно поведение.

При изчисляването на най-близките обекти беше необходимо правилно сравняване на координати и сортиране по разстояние. Реализацията решава това чрез отделна функция за изчисляване на разстоянието.

Работата с изображения също изискваше внимание, защото не всеки URL се зарежда еднакво добре в мобилно приложение. Затова са използвани директни връзки към изображения.

12. Заключение

The 100 National Tourist Sites представлява завършено мобилно приложение, което комбинира каталог на туристически обекти, детайлни страници, любими, посетени, прогрес, GPS функционалност, най-близки обекти и интеграция с Google Maps.

Проектът показва реална употреба на .NET MAUI за Android приложение. Използвани са локални данни, service класове, Preferences за персистентност, Geolocation API и практическа навигация между екрани.

В бъдеще приложението може да бъде разширено с пълния списък на всички 100 обекта, търсене, филтриране по категории, карта с всички локации, офлайн режим и cloud синхронизация. В текущия си вид то изпълнява основните изисквания за курсова работа по програмиране за мобилни устройства.

13. Литература

19. Microsoft Learn – .NET MAUI documentation.
20. Microsoft Learn – .NET MAUI Geolocation.
21. Microsoft Learn – .NET MAUI Preferences.
22. Microsoft Learn – Android permissions.
23. Google Maps documentation.
24. Официална информация за 100-те национални туристически обекта.
25. GitHub repository на проекта: <https://github.com/PetarIvanov33/The-100-National-Tourist-Sites>